

Kubernetes Workloads

1. Workload Types

Question: *What am I running?*

Defines the type of application or task Kubernetes should manage.

- **Pod** — Smallest deployable unit containing one or more containers.
 - **ReplicaSet** — Maintains a desired number of Pod replicas.
 - **Deployment** — Manages stateless applications and rolling updates.
 - **StatefulSet** — Manages stateful applications with stable identities and storage.
 - **DaemonSet** — Runs a Pod on every node (or selected nodes).
 - **Job** — Executes a task until it completes successfully.
 - **CronJob** — Executes Jobs on a schedule.
-

2. Deployment & Release Strategies

Question: *How do I release new versions safely?*

Defines how application changes are rolled out to users.

- **Recreate** — Stops old Pods before starting new Pods.
 - **Rolling Update** — Gradually replaces old Pods with new ones.
 - **Blue/Green** — Switches traffic between two environments.
 - **Canary** — Exposes a small percentage of traffic to a new version first.
 - **A/B Testing** — Routes different user groups to different versions.
-

3. Resource Management

Question: *How much infrastructure do I need and how should it be allocated?*

Controls resource allocation, limits, and scaling.

Planning & Design

Question: *How much CPU and memory should workloads consume?*

- **Requests** — Minimum resources guaranteed to a Pod.
- **Limits** — Maximum resources a Pod can consume.
- **QoS Classes** — Defines resource-pressure behavior.
- **ResourceQuota** — Restricts namespace-wide resource consumption.
- **LimitRange** — Applies default and maximum/minimum resource policies.

Scalability

Question: *How do workloads grow and shrink with demand?*

- **HPA** — Scales the number of Pods.
 - **VPA** — Adjusts Pod resource allocations.
 - **Cluster Autoscaler** — Adds or removes nodes.
 - **KEDA** — Scales based on external events.
-

4. Placement

Question: *Where should workloads run?*

Determines the best location for workloads within the cluster.

- **Scheduler** — Selects a node for each Pod.
 - **Node Selector** — Restricts Pods to specific nodes.
 - **Node Affinity** — Expresses advanced node placement preferences.
 - **Pod Affinity** — Places related Pods together.
 - **Pod Anti-Affinity** — Separates Pods from each other.
 - **Taints & Tolerations** — Controls which Pods can use specific nodes.
 - **Topology Spread Constraints** — Distributes Pods across nodes or zones.
-

5. Prioritization

Question: *Which workloads are most important?*

Determines how Kubernetes behaves when resources are insufficient.

- **PriorityClass** — Assigns importance levels to Pods.
 - **Preemption** — Removes lower-priority Pods to make room for higher-priority Pods.
-

6. Disruptions

Question: *What happens when workloads must be stopped, moved, or interrupted?*

Manages workload interruptions caused by maintenance, failures, or resource pressure.

- **Eviction** — Removes Pods from nodes.
 - **Pod Disruption Budget (PDB)** — Limits how many Pods can be voluntarily disrupted.
 - **Node Drain** — Safely removes workloads from a node before maintenance.
 - **Graceful Termination** — Allows applications time to shut down cleanly.
 - **Node Pressure Eviction** — Removes Pods during memory, disk, or PID pressure.
-

Complete Mental Model

Kubernetes Workloads

1. Workload Types

Question: What am I running?

2. Deployment & Release Strategies

Question: How do I release new versions safely?

3. Resource Management

Question: How much infrastructure do I need and how should it be allocated?

└─ Planning & Design

| Question: How much CPU and memory should workloads consume?

└─ Scalability

Question: How do workloads grow and shrink with demand?

4. Placement

Question: Where should workloads run?

5. Prioritization

Question: Which workloads are most important?

6. Disruptions

Question: What happens when workloads must be stopped, moved, or interrupted?

This sequence follows the lifecycle of operating workloads in Kubernetes: What → How to Deploy → How Much Resource → Where → Importance → What Happens During Interruptions.